

DBSCAN - UDEMY

Density-based spatial clustering of applications with noise

- It is a powerful tool which can be used for clustering and outlier detection.

DBSCAN key ideas :

- DBSCAN focuses on using density of points as its main factor for assigning cluster labels.
- This creates the ability to find cluster segmentations that other algorithms have difficulty.
- DBSCAN iterates through points and uses two key hyperparameters (epsilon and minimum number of points) to assign cluster labels.
- Unlike K-Means, it focuses on density as the main factor for cluster assignment of points.

DBSCAN Key Hyperparameters :

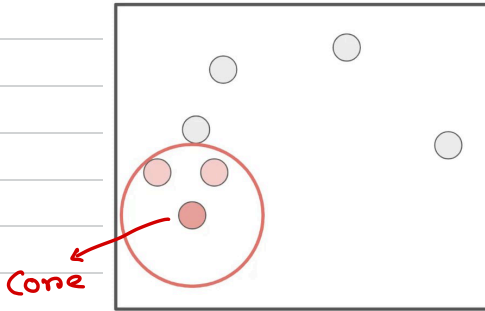
- **Epsilon:** Distance extended from a point.
- **Minimum Number of Points:** minimum no. of points in an epsilon distance.

DBSCAN Point Types:

- Core
- Border
- Outlier

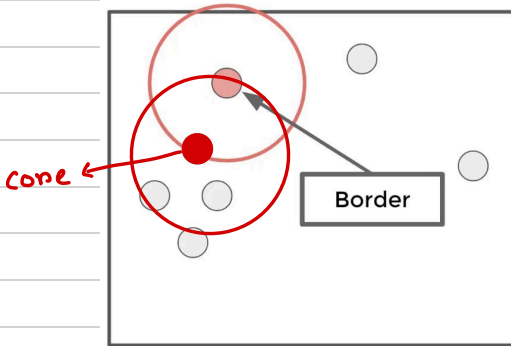
- **Cone:** points win min points in epsilon range. (including itself)

$\epsilon = 1$ and Min Points = 2



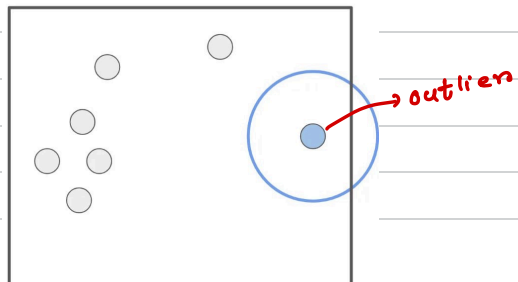
- **Border:** In epsilon range of core point, but does not contain min. no. of points.

$\epsilon = 1$ and Min Points = 3



- **Outlier:** Can not "reached" by points in a cluster assignment.

$\epsilon = 1$ and Min Points = 3



DBSCAN Procedure:-

- Pick a random point not yet assigned.
- Determine the point type.
- Once a core point has been found, add all directly reachable points to the same cluster as core.
- Repeat until all points have been assigned to a cluster or an outlier.

$\epsilon \uparrow$ and min no. of points $\downarrow \Rightarrow$ more no. of points in same cluster.



Hierarchical Clustering

Hierarchical Clustering

22 December 2023 11:41

Hierarchical clustering is a method of cluster analysis used in data mining. It seeks to build a hierarchy of clusters in a step-by-step manner. There are two main types of hierarchical clustering:

1. Agglomerative (Bottom-Up Approach):

- **Initial Step:** Starts by treating each data point as a separate cluster. So, if there are N data points, you begin with N clusters.
- **Clustering Process:** In each step, the algorithm merges the two clusters that are closest to each other until all the clusters are merged into one big cluster containing all data points.
- **Dendrogram:** The result can be represented in a tree-like structure called a dendrogram, which shows the arrangement of the clusters and their proximity.

2. Divisive (Top-Down Approach):

- **Initial Step:** Begins with all data points in a single cluster.
- **Clustering Process:** At each step, the algorithm splits the cluster until each cluster contains only one data point.
- **Top-Down Splitting:** This is less common compared to agglomerative clustering and is computationally more intensive.

Algorithm

22 December 2023 08:57

1. Initialization:

- Treat each data point as a separate cluster. Thus, if you have N data points, you start with N clusters, each containing just one data point.

2. Compute Distance Matrix:

- Calculate the distance between each pair of clusters. Common distance metrics include Euclidean, Manhattan, and Cosine distances. The choice of distance metric can significantly affect the outcome of the clustering.
- This results in an $N \times N$ distance matrix, where the distance between a cluster and itself is zero.

3. Find the Closest Clusters:

- Identify the two clusters that are closest to each other based on the distance matrix.

4. Merge Clusters:

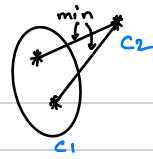
- Combine the two closest clusters into a single cluster.
- This step reduces the total number of clusters by one.

5. Update Distance Matrix:

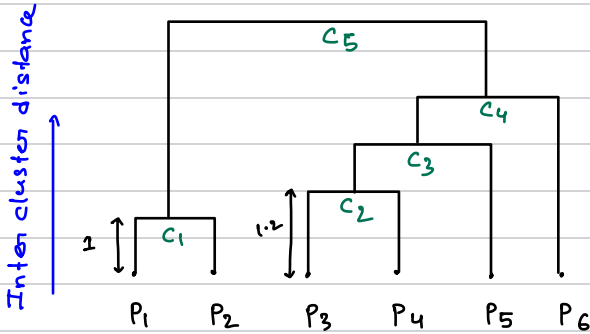
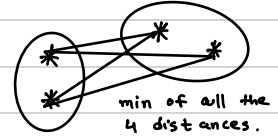
- Recalculate the distances between the new cluster and all the existing clusters.
- The method of recalculating the distance depends on the linkage criterion used.

Common linkage criteria include:

- o Single Linkage: Distance between two clusters is defined as the shortest distance between any two points in the clusters.
- o Complete Linkage: Distance is the longest distance between any two points in the clusters.
- o Average Linkage: Distance is the average distance between all pairs of points in the clusters.
- o Ward's Method: Distance is calculated as the increase in the total within-cluster variance after merging the clusters.



distance (C_1, P_3) is calculated using linkage.

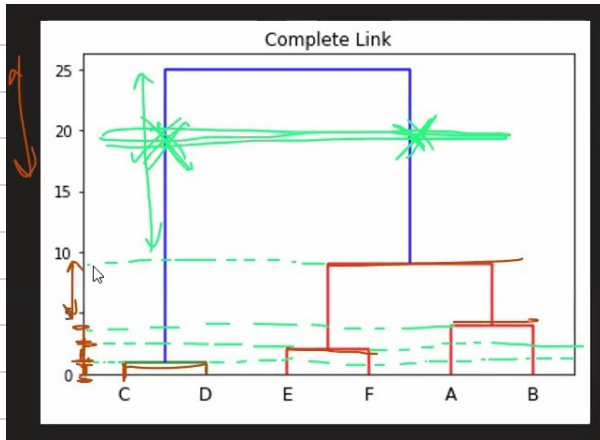


6. we repeat the steps until all data points merged into a single cluster.

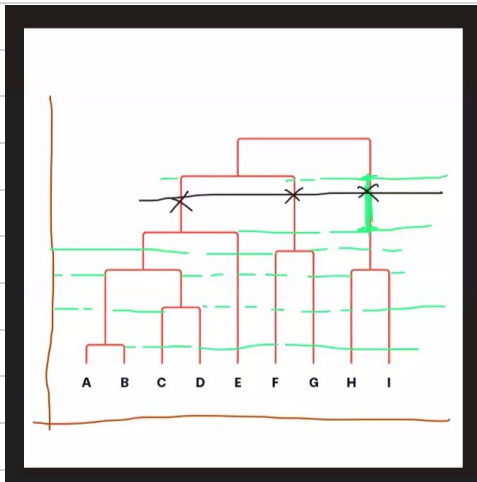
Note:-

- 1) Easy to understand and visualize.
- 2) Helps users decide how many clusters to choose.
- 3) Not necessary to choose cluster amount before the running the algorithm.

Finding n-clusters :



No. of clusters = 2



No. of clusters = 3

Linkage :-

In hierarchical clustering, linkage is the criterion that determines the distance between sets of observations as a function of the pairwise distances between observations. It's essentially the algorithm used to decide the proximity of clusters. There are several linkage methods, each defining the distance between clusters differently:

1. Single Linkage (Nearest Point Algorithm):

- The distance between two clusters is defined as the shortest distance from any member of one cluster to any member of the other cluster.
- Capable of detecting non-elliptical shapes in the data.
- Works well for datasets where the clusters are well-separated.
- May not perform well when clusters are close together or overlap as it is sensitive to outliers.

2. Complete Linkage (Farthest Point Algorithm):

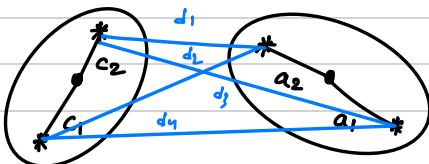
- The distance between two clusters is defined as the longest distance from any member of one cluster to any member of the other cluster.
- Less susceptible to noise and outliers compared to single linkage.
- Can struggle with elongated clusters or non-convex shapes.

3. Average Linkage (Average Link Algorithm):

- The distance between two clusters is defined as the average distance between each member of one cluster to every member of the other cluster.

4. Ward's Method:

- Objective: The main goal of Ward's method is to find the pair of clusters that, when merged, will increase the total within-cluster variance as little as possible. This is like trying to keep the clusters as compact as possible.
- Within-Cluster Variance: This is a measure of how spread out the points are within a cluster. A lower within-cluster variance means the points are closer to each other, and therefore, the cluster is more compact.
- How it Works: At each step of the algorithm, Ward's method looks at all possible pairs of clusters and calculates how much the within-cluster variance would increase if those two clusters were merged. It then merges the two clusters that result in the smallest increase in variance.
- Resulting Clusters: Because Ward's method tries to keep the within-cluster variance low, it tends to create clusters that are compact and roughly spherical in shape. This can be particularly effective if the natural groups in your data are also compact and spherical.



$$d_1^2 + d_2^2 + d_3^2 + d_4^2 - c_1^2 - c_2^2 - a_1^2 - a_2^2$$

Space Complexity :-

$\frac{n(n-1)}{2}$ distances need to be stored.

$$O(n^2)$$

Time Complexity :- $O(n^2 \log n)$ to $O(n^3)$

Agglomerative Clustering (Bottom-up)

This is the more common approach to hierarchical clustering. The time complexity is influenced by the method used to determine the closest pair of clusters at each step.

- **Naive Approach:** $O(n^3)$
A straightforward implementation involves computing the $n \times n$ distance matrix ($O(n^2)$) and then, in each of the $n-1$ steps, searching the entire matrix for the minimum distance to merge. This repeated search leads to a cubic time complexity.
- **Optimized Approach:** $O(n^2 \log n)$
The performance can be improved by using a priority queue (or a heap) to store the distances between clusters. This allows for finding the minimum distance more efficiently. While the initial distance matrix calculation is still $O(n^2)$, subsequent updates and retrievals from the priority queue lead to a time complexity of $O(n^2 \log n)$.
- **Specialized Algorithms for Specific Linkages:** $O(n^2)$
For certain linkage criteria, even more efficient algorithms exist:
 - **Single Linkage:** The SLINK algorithm can achieve a time complexity of $O(n^2)$.
 - **Complete Linkage:** The CLINK algorithm can also run in $O(n^2)$ time.

Advantages

1. **Discovery of Hierarchical Structure:** The algorithm reveals the hierarchy and nested structure within the data, which can be informative for understanding complex relationships.
2. **Useful for Any Distance Measure:** The method can be used with any distance measure, which is beneficial for different types of data, such as genomic data or mixed data types.
3. **Does Not Assume Clusters as Spherical:** Unlike K-means, agglomerative clustering does not assume that clusters are spherical in shape, which can result in more natural cluster shapes.
4. **Easy to Implement and Understand:** The algorithm is conceptually simple and can be easily implemented, making it accessible for users with varying levels of expertise.
5. **Robust to Noise and Outliers:** With the appropriate choice of linkage criteria (such as Ward's method), hierarchical clustering can be relatively robust to noise and outliers, as these will typically be merged into clusters at later stages of the process.

Disadvantages

1. **Computational Complexity:** One of the biggest drawbacks is its computational cost. The algorithm has a time complexity of $O(n^3)$ and space complexity of $O(n^2)$ for the simplest implementations, making it impractical for large datasets.
2. **Sensitivity to Noise and Outliers:** Certain linkage criteria, such as single linkage, can be highly sensitive to noise and outliers, which can lead to misleading results. Outliers can cause clusters to merge prematurely, distorting the true structure of the data.
3. **Difficulty in Identifying the Number of Clusters:** While the dendrogram can provide insights into the potential number of clusters, there is often subjectivity involved in interpreting where to 'cut' the dendrogram to define the clusters.
4. **Arbitrary Decisions in Linkage Criteria:** The choice of linkage criteria (single, complete, average, Ward's, etc.) can significantly affect the results, and there is no definitive rule for choosing the best method, which can make the process somewhat arbitrary.
5. **No Global Objective Function:** Unlike K-means, which minimizes within-cluster variance, there's no clear global objective in hierarchical clustering, which can make it difficult to assess the quality of the resulting clusters.

K Means Clustering

Defⁿ:- K-Means is an unsupervised machine learning algorithm used for partitioning a dataset into a pre-determined number of clusters, denoted by 'k'. It is a centroid-based algorithm where goal is to group similar data points together and discover underlying.

The main objective of K-Means algorithm is to minimize the within-cluster sum of squares (WCSS), also known as inertia.

This is the sum of squared distances between each data point and its assigned cluster's centroid.

$$WCSS = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

steps:-

The algorithm works iteratively in four basic steps:

1. **Initialization:** First, you must choose the number of clusters, K . Then, K initial centroids are chosen. A common method is to pick K data points from the dataset at random. A smarter method is using **K-Means++**, which initializes centroids to be far apart from each other.
2. **Assignment Step:** Each data point in the dataset is assigned to the **nearest centroid**. The "nearness" is typically measured using the Euclidean distance.
3. **Update Step:** After all data points have been assigned to a cluster, the centroid of each cluster is recalculated. The new centroid is the **mean** of all data points belonging to that cluster.
4. **Convergence:** Steps 2 and 3 are repeated until a stopping criterion is met. This happens when:
 - The centroids no longer move significantly between iterations.
 - The cluster assignments for the data points no longer change.
 - A maximum number of iterations is reached.

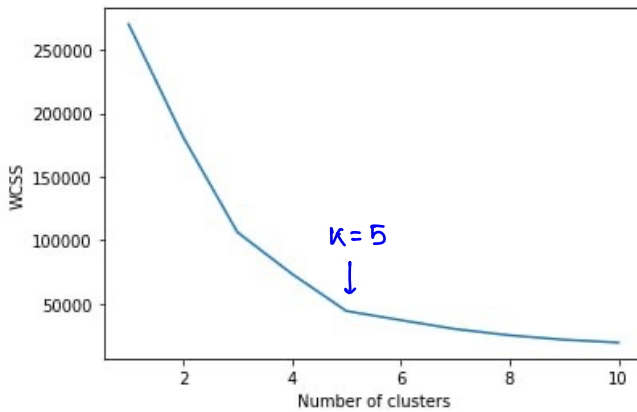
At the end of this process, the data is partitioned into K distinct clusters.

Choosing K - Value :-

Elbow Method

A: The most common method is the **Elbow Method**.

- **Process:** Run K-Means for a range of K values (e.g., 1 to 10) and plot the WCSS (inertia) against K.
- **The "Elbow":** The plot will show a sharp decrease in WCSS as K increases initially. The "elbow" is the point where the rate of decrease sharply bends and becomes more gradual. The K at this bend is often a good choice.
- **Limitation:** The elbow is not always clear and can be subjective.



Silhouette Score :-

The silhouette score measures the quality of a clustering configuration.

For a single point:

$$s(i) = \frac{(b(i) - a(i))}{\max(b(i), a(i))}$$

$a(i)$: Mean distance between point i and all other

points in same cluster.

$b(i)$: Mean distance between point i and all points in the nearest cluster.

- **Interpretation:** $s(i)$ ranges from -1 to 1.
 - **+1:** The point is well-matched to its own cluster and poorly matched to neighboring clusters.
 - **0:** The point is on the boundary between two clusters.
 - **-1:** The point is likely assigned to the wrong cluster.
- The average silhouette score over all points is used to evaluate the entire clustering.

- You calculate the average Silhouette Score for all data points for different values of K . The value of K that maximizes the average Silhouette Score is chosen as the optimal one.

Q6: Why is the choice of the initial centroids so important?

A: K-Means is sensitive to initial centroid placement because it can converge to a local minimum of the WCSS, rather than the global minimum. Poor initialization can lead to:

- **Sub-optimal Clustering:** The final clusters may not be the most natural grouping of the data.
- **Slow Convergence:** The algorithm may take more iterations to stabilize.
- **Empty Clusters:** In some cases, a centroid might not be assigned any data points.

Q7: How does K-Means++ address the initialization problem?

A: K-Means++ is a smart initialization technique that spreads out the initial centroids:

1. Randomly select the first centroid from the data points.
2. For each subsequent centroid:
 - a. Compute the squared distance $D(x)^2$ from each data point to the nearest already chosen centroid.
 - b. Select the next centroid from the data points with a probability proportional to $D(x)^2$. Points farther from existing centroids have a higher chance of being selected.

This method ensures that the initial centroids are far apart from each other, leading to better, more consistent, and faster convergence.

Q5: How is K-Means different from other clustering algorithms like DBSCAN or Hierarchical Clustering?

Feature	K-Means	Hierarchical Clustering	DBSCAN
Number of Clusters	Must be specified beforehand (K).	Not required beforehand. Chosen by cutting the dendrogram.	Not required. Discovered by the algorithm.
Cluster Shape	Assumes spherical, equally-sized clusters.	Can handle any shape, but depends on linkage method.	Can find arbitrarily shaped clusters.
Outlier Handling	Poor. Forces all points into a cluster.	Poor. Outliers are eventually merged into a cluster.	Excellent. Robustly identifies and separates outliers.
Scalability	High. Very efficient for large datasets.	Low. Computationally expensive ($O(n^2)$ or more).	High ($O(n \log n)$ on average).
Determinism	Non-deterministic due to random initialization.	Deterministic. Always produces the same result.	Deterministic (mostly, except for border points).
Primary Use Case	Customer segmentation, document clustering.	Taxonomy analysis (e.g., biology), social networks.	Anomaly detection, geospatial data analysis.

Advantages 🍌

- **Simplicity and Speed:** It's very easy to understand and implement. Its time complexity is $O(n \cdot K \cdot i \cdot d)$, which is linear in the number of data points (n), making it computationally efficient and fast for large datasets.
- **Scalability:** It scales well to large datasets, unlike hierarchical clustering methods which can be more computationally expensive.
- **Guaranteed Convergence:** The algorithm is guaranteed to converge to a local optimum.

Disadvantages 🍌

- **Need to Pre-specify K:** The number of clusters, K , must be chosen beforehand, which can be a challenge without domain knowledge.
- **Sensitivity to Initialization:** The final clustering results can vary depending on the initial placement of centroids. Using the **K-Means++** initialization method or running the algorithm multiple times with different random seeds can mitigate this.
- **Assumes Spherical Clusters:** K-Means implicitly assumes that clusters are spherical, have similar sizes, and similar densities. It performs poorly on clusters with complex, non-convex shapes or varying sizes.
- **Sensitivity to Outliers:** Since centroids are calculated as the mean of cluster points, they are highly sensitive to outliers, which can skew the centroids and lead to poor clustering results.

Q15: What preprocessing steps are crucial before applying K-Means?

A:

1. **Feature Scaling:** This is critical. K-Means is distance-based. If features are on different scales (e.g., age 0-100 vs. salary 50,000-200,000), the feature with the larger scale will dominate the distance calculation. Use **Standardization (Z-score)** or **Normalization (Min-Max)**.
2. **Handling Outliers:** Identify and remove/cap outliers as they can skew the centroids.
3. **Handling Missing Values:** Impute or remove missing data.

Optimization function for the K-means :-

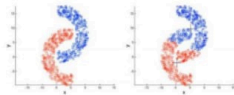
$$c_1, c_2, \dots, c_k = \operatorname{argmin} \sum_{i=1}^k \sum_{x \in c_i} \|x - c_i\|_2^2$$

18. Explain some cases where K means clustering fails to give good results.

The K means clustering algorithm fails to give good results in the below-mentioned cases:

- When the dataset contains **outliers**
- When the **density spread** of data points across the data space is different.
- When the data points follow a **non-convex shape**.

Non-convex/non-round-shaped clusters: Standard K-means fails!



Clusters with different densities

